STUDENT ATTENDANCE TRACKER

Level 2 Hackathon — Fundamentals of Web Development

Student Name	S. Mohammed Kaleem
Register Number	36725U18016
Department	B.Sc. Computer Science
Student ID	astvu36736725u18016
College	Global College of Arts and Science
Date of Submission	28 October 2026

Improved report with clearer explanations, application flow diagram, complete HTML/CSS/JavaScript example, and output preview.

GITHUB PROJECT LINK

https://github.com/kaleemfreefire9-del/Kaleem/tree/main

1. Problem Statement

Colleges need a dependable way to record daily student attendance, check who is present or absent, and calculate each student's attendance percentage. A paper register can take time to update, search, and calculate. This project demonstrates a browser-based Student Attendance Tracker that organises student records and attendance entries in one dashboard.

The application is designed as a learning project. It uses HTML for the page structure, CSS for the appearance, and JavaScript for validation, attendance calculations, search, and saving data in the browser. The example stores information in Local Storage on the same browser and device; it is not a shared college database.

2. Objectives

• Add a student by entering a name and student ID.

• Record a student's status as Present or Absent for a selected date.

• Calculate attendance percentage from the saved attendance entries.

• Search for students by name or ID.

• Show summary information such as total students, present count, absent count, and overall attendance.

• Prevent a second attendance entry for the same student on the same date.

• Keep saved data after refreshing the page by using browser Local Storage.

3. Architecture

The user starts at the dashboard, adds or selects a student, chooses an attendance date, and marks the status. JavaScript checks the input and checks for a duplicate entry. If valid, the entry is saved to Local Storage. The dashboard is then refreshed to show updated totals and percentages.

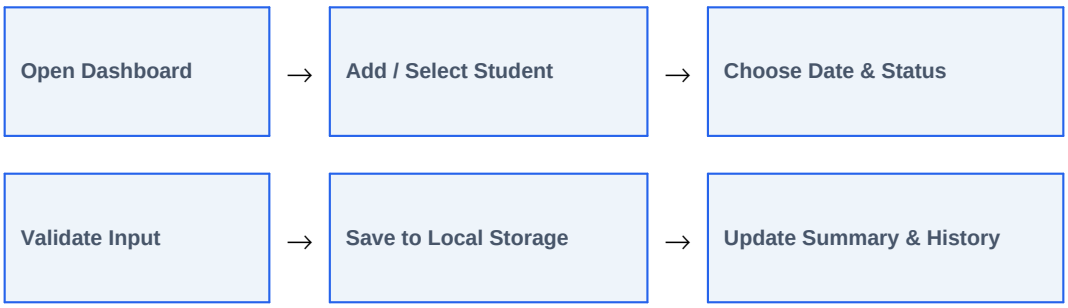


Figure 1. Application flow from user input to displayed result.

Technology roles

• **HTML:** creates headings, input fields, buttons, summary cards, and the attendance table.

• **CSS:** controls spacing, colours, responsive layout, table styling, and Present/Absent labels.

• **JavaScript:** validates form values, updates arrays, blocks duplicate entries, searches records, and calculates percentages.

• **Local Storage:** saves JSON data in the browser so records remain after a refresh on that same browser.

4. Practical Implementation

The interface is organised into a dashboard, a student entry form, an attendance section, a search box, and summary cards. When a user adds a student or records attendance, JavaScript updates the stored data and redraws the visible table. The browser's Local Storage API is used instead of a server, so the sample is suitable for

a small demonstration but does not synchronise between devices.

4.1 Features Implemented

Feature	How it works / purpose
Student entry	The user enters a name and student ID. Empty values are rejected.
Daily attendance	The user chooses a date and marks Present or Absent.
Percentage	Present entries ÷ all recorded entries × 100.
Search	Filters the table by a student's name or ID.
Summary cards	Displays total students, present, absent, and overall attendance.
Duplicate prevention	Checks student ID plus date before saving a new entry.
Local Storage	Persists students and attendance in the same browser.

4.2 Source Code

The following is a complete single-file demonstration. Save it as **index.html**, then open it in Chrome or another modern browser. It contains the HTML, CSS, and JavaScript needed for the basic tracker.

Complete HTML, CSS and JavaScript

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Student Attendance Tracker</title>
<style>
  * { box-sizing: border-box; }
  body { margin: 0; font-family: Arial, sans-serif;
         background: #f1f5f9; color: #1e293b; }
  header { background: #173b63; color: white; padding: 18px 5%; }
  main { max-width: 1100px; margin: 24px auto; padding: 0 16px; }
  .cards { display: grid; grid-template-columns: repeat(4, 1fr);
           gap: 12px; margin: 16px 0; }
  .card, .panel { background: white; border-radius: 12px;
                  padding: 16px; box-shadow: 0 2px 8px #dbe3ed; }
  .card strong { display: block; font-size: 24px; margin-top: 8px; }
  .layout { display: grid; grid-template-columns: 280px 1fr; gap: 16px; }
  input, select, button { width: 100%; padding: 10px; margin: 5px 0 10px;
                          border: 1px solid #cbd5e1; border-radius: 6px; }
  button { background: #2563eb; color: white; border: 0; cursor: pointer; }
  table { width: 100%; border-collapse: collapse; font-size: 14px; }
  th, td { padding: 10px; text-align: left; border-bottom: 1px solid #e2e8f0; }
  th { background: #edf2f7; }
  .present { color: #166534; background: #dcfce7; padding: 4px 8px; border-radius: 10px; }
  .absent { color: #991b1b; background: #fee2e2; padding: 4px 8px; border-radius: 10px; }
  @media (max-width: 700px) {
    .cards { grid-template-columns: repeat(2, 1fr); }
    .layout { grid-template-columns: 1fr; }
  }
</style>
</head>
<body>
<header><h1>Student Attendance Tracker</h1>
<p>Manage students and record daily attendance</p></header>
<main>
  <h2>Attendance Dashboard</h2>
  <div class="cards">
    <div class="card">Total Students<strong id="total">0</strong></div>
    <div class="card">Present Today<strong id="present">0</strong></div>
    <div class="card">Absent Today<strong id="absent">0</strong></div>
    <div class="card">Overall Attendance<strong id="overall">0%</strong></div>
  </div>
  <div class="layout">
    <section class="panel">
      <h3>Add Student</h3>
      <form id="studentForm">
        <label for="studentName">Student name</label>
        <input id="studentName" required placeholder="Enter student name">
        <label for="studentId">Student ID</label>
        <input id="studentId" required placeholder="Enter unique ID">
        <button type="submit">Add Student</button>
      </form>
      <label for="date">Attendance date</label>
      <input id="date" type="date" required>
      <label for="search">Search by name or ID</label>
      <input id="search" placeholder="Type to search">
    </section>
    <section class="panel">
      <h3>Attendance Records</h3>
      <p id="message" aria-live="polite"></p>
      <div style="overflow-x:auto">
        <table>
          <thead><tr><th>Student</th><th>ID</th><th>Status</th><th>Attendance</th><th>Action</th></tr></thead>
          <tbody id="rows"></tbody>
        </table>
      </div>
    </section>
  </div>
</main>
<script>
const load = key => JSON.parse(localStorage.getItem(key) || "[]");
let students = load("students");
let attendance = load("attendance");
const today = new Date().toISOString().slice(0, 10);
document.getElementById("date").value = today;

function persist() {
  localStorage.setItem("students", JSON.stringify(students));
```

```
  localStorage.setItem("attendance", JSON.stringify(attendance));
}
function percent(id) {
  const entries = attendance.filter(a => a.studentId === id);
  if (!entries.length) return 0;
  const presentCount = entries.filter(a => a.status === "Present").length;
  return Math.round(presentCount / entries.length * 100);
}
function mark(id, status) {
  const date = document.getElementById("date").value;
  if (!date) { alert("Please choose a date."); return; }
  if (attendance.some(a => a.studentId === id && a.date === date)) {
    alert("Attendance already exists for this student on this date.");
    return;
  }
  attendance.push({ studentId: id, date, status });
  persist(); render();
}
function render() {
  const date = document.getElementById("date").value;
  const query = document.getElementById("search").value.toLowerCase();
  const todays = attendance.filter(a => a.date === date);
  document.getElementById("total").textContent = students.length;
  document.getElementById("present").textContent =
    todays.filter(a => a.status === "Present").length;
  document.getElementById("absent").textContent =
    todays.filter(a => a.status === "Absent").length;
  const totalEntries = attendance.length;
  const presentEntries = attendance.filter(a => a.status === "Present").length;
  document.getElementById("overall").textContent =
    (totalEntries ? Math.round(presentEntries / totalEntries * 100) : 0) + "%";

  const visible = students.filter(s =>
    (s.name + " " + s.id).toLowerCase().includes(query));
  document.getElementById("rows").innerHTML = visible.map(s => {
    const status = todays.find(a => a.studentId === s.id)?.status || "Not marked";
    return `<tr><td>${escapeHTML(s.name)}</td><td>${escapeHTML(s.id)}</td>
      <td>${status === "Present" ? '<span class="present">Present</span>' :
      status === "Absent" ? '<span class="absent">Absent</span>' : status}</td>
      <td>${percent(s.id)}%</td><td>
      <button type="button" onclick="mark('${safeId(s.id)}','Present')">Present</button>
      <button type="button" onclick="mark('${safeId(s.id)}','Absent')">Absent</button>
      </td></tr>`;
  }).join("");
}
function escapeHTML(value) {
  return String(value).replace(/[&<>"']/g, ch =>
    ({'&':'&amp;','<':'&lt;','>':'&gt;','"':'&quot;',"'":'&#39;'}[ch]));
}
function safeId(value) {
  return String(value).replace(/\\/g, "\\\\").replace(/'/g, "\\'");
}
document.getElementById("studentForm").addEventListener("submit", event => {
  event.preventDefault();
  const name = document.getElementById("studentName").value.trim();
  const id = document.getElementById("studentId").value.trim();
  if (!name || !id) { alert("Enter both name and ID."); return; }
  if (students.some(s => s.id.toLowerCase() === id.toLowerCase())) {
    alert("That student ID already exists."); return;
  }
  students.push({ name, id });
  persist(); event.target.reset(); render();
});
document.getElementById("search").addEventListener("input", render);
document.getElementById("date").addEventListener("change", render);
render();
</script>
</body>
</html>
```

Code Explanation

HTML — structure

HTML defines the dashboard title, summary cards, student form, date field, search field, and table. The **studentForm** receives the name and ID. The table body with id **rows** is the area JavaScript fills with student records.

```
<form id="studentForm">
  <input id="studentName" required placeholder="Enter student name">
  <input id="studentId" required placeholder="Enter unique ID">
  <button type="submit">Add Student</button>
</form>
<table>
  <thead><tr><th>Student</th><th>ID</th><th>Status</th></tr></thead>
  <tbody id="rows"></tbody>
</table>
```

CSS — appearance and responsive layout

CSS sets the font, page background, cards, table borders, button colour, and status labels. The media query changes the layout on smaller screens so the form and table are easier to use on a phone.

```
.cards { display: grid; grid-template-columns: repeat(4, 1fr); gap: 12px; }
.present { color: #166534; background: #dcfce7; }
.absent { color: #991b1b; background: #fee2e2; }
@media (max-width: 700px) {
  .cards { grid-template-columns: repeat(2, 1fr); }
  .layout { grid-template-columns: 1fr; }
}
```

JavaScript — logic and storage

JavaScript reads previously saved JSON arrays from Local Storage. The **mark()** function checks whether the student already has an entry for the chosen date. If an entry exists, it stops and shows a message; otherwise it saves the new status and refreshes the dashboard.

```
function mark(id, status) {
  const date = document.getElementById("date").value;
  if (attendance.some(a => a.studentId === id && a.date === date)) {
    alert("Attendance already exists for this student on this date.");
    return;
  }
  attendance.push({ studentId: id, date, status });
  localStorage.setItem("attendance", JSON.stringify(attendance));
  render();
}
```

The percentage function counts all entries for one student and counts how many are marked Present. It then divides present entries by total entries and multiplies by 100.

```
function percent(id) {
  const entries = attendance.filter(a => a.studentId === id);
  if (!entries.length) return 0;
  const presentCount = entries.filter(a => a.status === "Present").length;
  return Math.round(presentCount / entries.length * 100);
}
```

4.3 Project Execution

• Open a text editor such as Notepad or VS Code.

• Copy the complete source code from Section 4.2 and save it as index.html.

• Open index.html in a modern web browser.

• Enter a student name and unique ID, then select Add Student.

• Choose an attendance date and click Present or Absent beside a student.

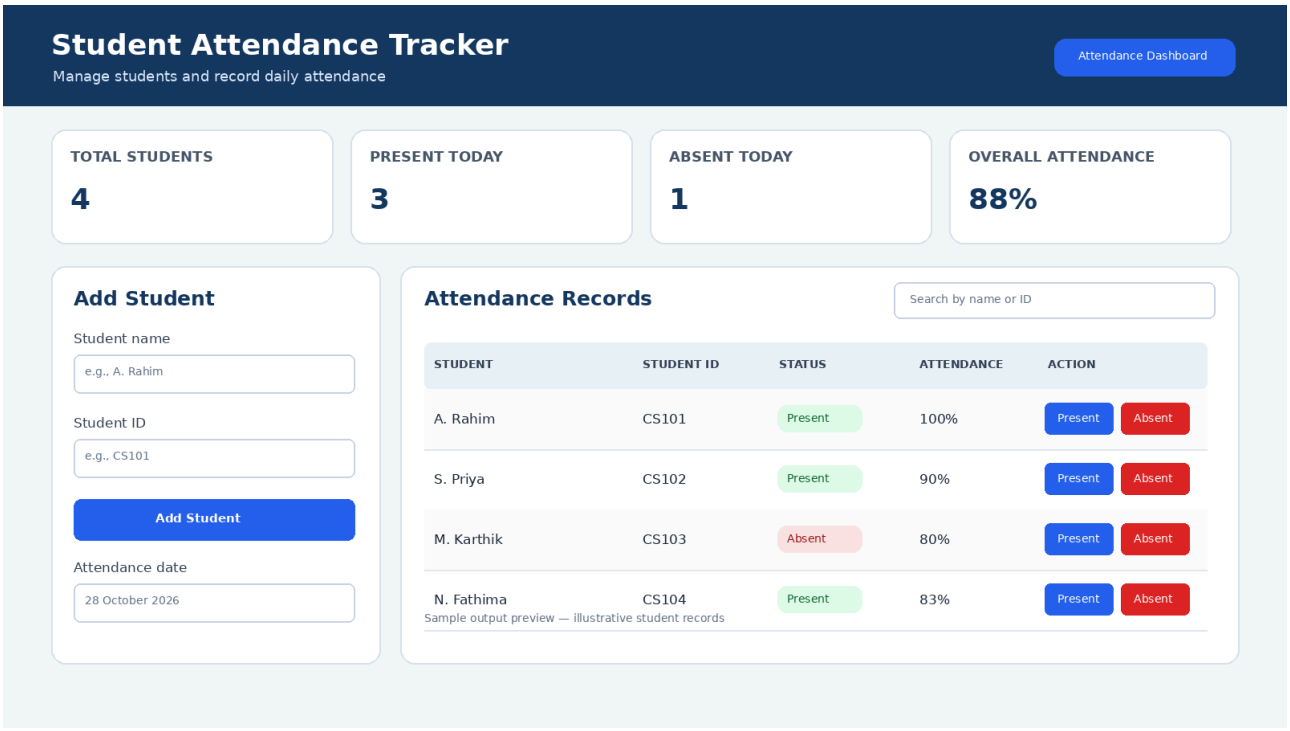
• Check the summary cards and attendance percentage. Use the search box to find a student.

• Refresh the browser and confirm that records remain saved in Local Storage.

Important: The sample stores data only in the current browser profile. Clearing browser site data can delete the saved records. A real college system should use authentication, access control, backups, and a secure database.

5. Output Screenshot / Interface Preview

The image below is the dashboard preview included in the earlier report. It illustrates summary cards, a student entry form, a search field, and a table with sample attendance statuses. The sample values are illustrative and are not actual college records.



Expected output: after adding a student and marking attendance, the student appears in the table. The Present/Absent totals and percentage update automatically. Duplicate attendance for the same student and date is blocked.

6. Technical Explanation

Concept	Explanation
DOM	JavaScript accesses page elements using document.getElementById and updates their text or table rows.
Arrays and objects	students and attendance are arrays; each item is an object containing a student ID, date, and status.
Event handling	Form submission, search input, and date changes trigger JavaScript functions.
Validation	Required fields and duplicate checks prevent incomplete or repeated records.
Local Storage	JSON.stringify converts arrays to text for saving; JSON.parse restores the data when the page opens.
Percentage	(Number of Present entries ÷ total recorded entries) × 100.

7. Conclusion

The Student Attendance Tracker demonstrates how HTML, CSS, and JavaScript can work together to build an interactive web application. The project supports student entry, daily attendance marking, searching, percentage calculation, duplicate prevention, and browser-based storage. It also provides practice with forms, DOM updates, arrays, events, conditions, and JSON data.

8. Future Enhancements

• Add secure login for teachers and administrators.

• Use a database and server so records can be shared across authorised devices.

• Add edit and delete options with confirmation.

• Create downloadable PDF and Excel attendance reports.

• Add low-attendance alerts and charts.

• Add classes, departments, semesters, and multiple sessions per day.